FULL NAME: RUSUMBANYA MBONI

STUDENT ID: ST10489107

QUALIFICATION: Diploma in Software Development (DISD)

YEAR: 2nd

SUBJECT: Programming (PROG 6212)

ASSIGNMENT_1

SECTION A: Entity Relationship Diagram (ERD) Design

Step 1 — Identify Database Entities and Attributes

The RaceDay system requires a relational database design that supports user management, event creation, event categories, participant enrolments, and race results. The following entities were identified based on the functional requirements of the system.

1. Role Entity

Purpose:

The Role entity stores the different user roles available in the RaceDay system. The system supports two roles: Organiser and Participant.

Attributes:

Attribute	Data Type	Constraint
RoleId	INT	Primary Key (pk)
RoleName	VARCHAR(50)	Unique, Not Null

Relationship:

Role 1 : Many User

One role can be assigned to many users, while each user has one assigned role.

2. User Entity

Purpose:

The User entity stores account and profile information for both Organisers and Participants.

Attributes:

Attribute	Data Type	Constraint
UserId	INT	Primary Key (pk)
RoleId	INT	Foreign Key (FK)
FirstName	VARCHAR(100)	Not Null
LastName	VARCHAR(100)	Not Null
Email	VARCHAR(255)	Unique, Not Null
PasswordHash	VARCHAR(255)	Not Null
PhoneNumber	VARCHAR(20)	Nullable

Attribute	Data Type	Constraint
DateOfBirth	DATE	Nullable
CreatedAt	DATETIME	Not Null

Foreign Key:

RoleId references Role(RoleId)

Relationships:

- **Role 1 : Many User**
- **User (Organiser) 1 : Many Event**
- **User (Participant) 1 : Many Enrolment**

3. Event Entity

Purpose:

The Event entity stores information about running, walking, and cycling events created by Organisers.

Attributes:

Attribute	Data Type	Constraint
EventId	INT	Primary Key
OrganiserId	INT	Foreign Key
EventName	VARCHAR(150)	Not Null
Description	VARCHAR(1000)	Nullable
EventDate	DATETIME	Not Null
Location	VARCHAR(255)	Not Null
Distance	DECIMAL(6,2)	Not Null
EventType	VARCHAR(20)	Not Null
CreatedAt	DATETIME	Not Null

Foreign Key:

OrganiserId references User(UserId)

Relationship:

User (Organiser) 1 : Many Event

One Organiser can create multiple events.

4. Category Entity

Purpose:

The Category entity stores the available participation categories for each event.

Examples include:

- Under 20
- Senior
- 10km
- 21km

Attributes:

Attribute	Data Type	Constraint
CategoryId	INT	Primary Key
EventId	INT	Foreign Key
CategoryName	VARCHAR(100)	Not Null
MinimumAge	INT	Nullable
MaximumAge	INT	Nullable
Distance	DECIMAL(6,2)	Not Null

Foreign Key:

EventId references Event(EventId)

Relationship:

Event 1 : Many Category

One event can contain multiple categories.

5. Enrolment Entity

Purpose:

The Enrolment entity records participants entering events and the selected category. It resolves the many-to-many relationship between Participants and Events.

Attributes:

Attribute	Data Type	Constraint

Attribute	Data Type	Constraint
EnrolmentId	INT	Primary Key
ParticipantId	INT	Foreign Key
EventId	INT	Foreign Key
CategoryId	INT	Foreign Key
EnrolmentDate	DATETIME	Not Null
Status	VARCHAR(30)	Not Null

Foreign Keys:

- ParticipantId references User(UserId)
- EventId references Event(EventId)
- CategoryId references Category(CategoryId)

Relationships:

- **User (Participant) 1 : Many Enrolment**
- **Event 1 : Many Enrolment**
- **Category 1 : Many Enrolment**

6. Result Entity

Purpose:

The Result entity stores participant performance information after completing an event.

Attributes:

Attribute	Data Type	Constraint
ResultId	INT	Primary Key
EnrolmentId	INT	Foreign Key
FinishTime	TIME	Not Null
FinishPosition	INT	Not Null
RecordedAt	DATETIME	Not Null

Foreign Key:

EnrolmentId references Enrolment(EnrolmentId)

Relationship:

Enrolment 1 : 0--1 Result

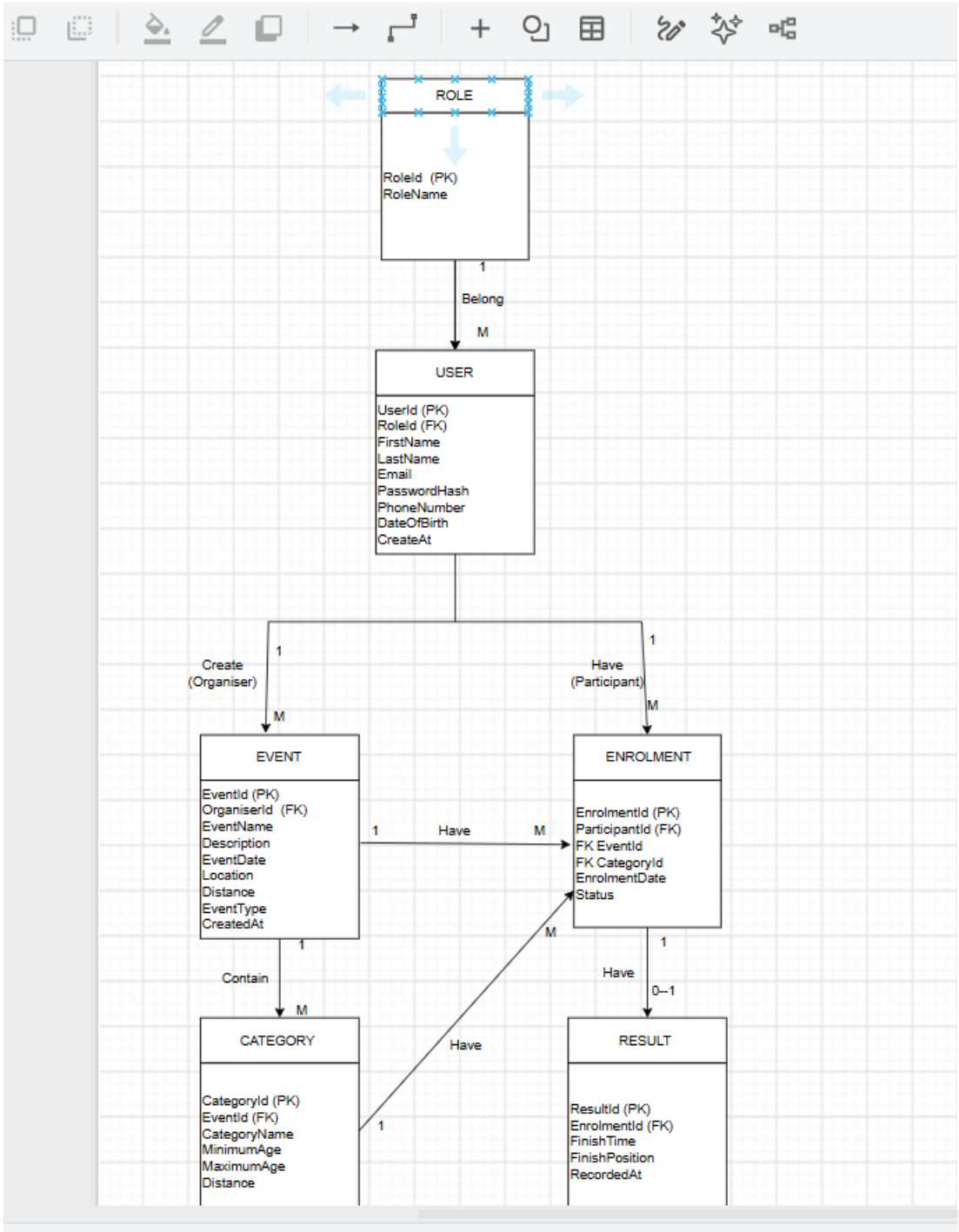
One enrolment can have zero or one result.

Relationship Summary

Relationship	Cardinality	Description
Role → User	1 : Many	One role can belong to many users
User (Organiser) → Event	1 : Many	One organiser can create many events
Event → Category	1 : Many	One event can contain many categories
User (Participant) → Enrolment	1 : Many	One participant can have many enrolments
Event → Enrolment	1 : Many	One event can have many enrolments
Category → Enrolment	1 : Many	One category can have many enrolments
Enrolment → Result	1 : 0--1	An enrolment may have no result or one result

Database Design Notes: The Enrolment entity is used as a junction table to resolve the many-to-many relationship between Participants and Events. This design supports authentication, role-based access, event management, participant registration, category selection, and result tracking required by the RaceDay system.

➢ Visual ERD Diagram



REFERENCE : I Used Draw.io for the Visual

ERD Diagram

Section B - API Endpoint Plan

RaceDay System

Functional Requirements Check

Requirement	How the API Meets It	Main Endpoint(s)
Authentication	Users can register and log in, with their role stored and returned after authentication.	/api/auth/register, /api/auth/login
User Profile	Organisers and Participants can view and update only their own profile information.	/api/profile
Events	Organisers can create, update and delete events. Both roles can view events. Events include name, description, date, location, distance and event type (run, walk or cycle).	/api/events, /api/events/{id}, /api/events/my-events
Categories	Organisers can define age or distance categories for events. Both roles can view available categories.	/api/events/{eventId}/categories, /api/categories/{id}
Event Enrolments	Participants enter an event by selecting a category. The system links Participant, Event and Category. Organisers can view enrolments for events they own.	/api/events/{eventId}/enrolments, /api/enrolments/my-enrolments
Results	Organisers capture finish times and finishing positions. Participants can view their own results.	/api/enrolments/{enrolmentId}/results, /api/results/my-results

1. Authentication Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new RaceDay account and records whether the user is an Organiser or Participant.	None (Public)	{ firstName, lastName, email, password, phoneNumber, dateOfBirth, roleId }	201 Created - account created. 400 Bad Request - invalid data. 409 Conflict - email already exists.
POST	/api/auth/login	Authenticates a registered user and returns authentication details together with the user role.	None (Public)	{ email, password }	200 OK - login successful and token/user details returned. 400 Bad Request - missing data. 401 Unauthorized - invalid email or password.

2. User Profile Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/profile	Returns the profile information of the currently logged-in user.	Organiser or Participant	None	200 OK - own profile returned. 401 Unauthorized - user is not logged in.
PUT	/api/profile	Updates the profile information of the currently logged-in user.	Organiser or Participant	{ firstName, lastName, phoneNumber, dateOfBirth }	200 OK - profile updated. 400 Bad Request - invalid data. 401 Unauthorized - user is not logged in.

3. Event Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Returns the list of RaceDay events that logged-in Organisers and Participants can view.	Organiser or Participant	None	200 OK - list of events returned. 401 Unauthorized - user is not logged in.
GET	/api/events/{id}	Returns the full details of one event.	Organiser or Participant	None	200 OK - event returned. 401 Unauthorized - user is not logged in. 404 Not Found - event does not exist.
POST	/api/events	Creates a new event and links it to the logged-in Organiser. eventType must be run, walk or cycle.	Organiser	{ eventName, description, eventDate, location, distance, eventType }	201 Created - event created. 400 Bad Request - invalid event data. 401 Unauthorized - not logged in. 403 Forbidden - user is not an Organiser.
PUT	/api/events/{id}	Updates an event owned by the logged-in Organiser.	Organiser	{ eventName, description, eventDate, location, distance, eventType }	200 OK - event updated. 400 Bad Request - invalid data. 401 Unauthorized - not logged in. 403 Forbidden - organiser does not own the event. 404 Not Found - event not found.
DELETE	/api/events/{id}	Deletes an event owned by the logged-in Organiser.	Organiser	None	204 No Content - event deleted. 401 Unauthorized - not logged in. 403 Forbidden - organiser does not own the event. 404 Not Found - event not found.
GET	/api/events/my-events	Returns only the events created by the currently logged-in Organiser.	Organiser	None	200 OK - organiser events returned. 401 Unauthorized - not logged in. 403 Forbidden - user is not an Organiser.

4. Category Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Returns the available age or distance categories for a selected event.	Organiser or Participant	None	200 OK - categories returned. 401 Unauthorized - not logged in. 404 Not Found - event not found.
POST	/api/events/{eventId}/categories	Creates an age or distance category for an event owned by the logged-in Organiser.	Organiser	{ categoryName, minimumAge, maximumAge, distance }	201 Created - category created. 400 Bad Request - invalid category data. 401 Unauthorized - not logged in. 403 Forbidden - organiser does not own the event. 404 Not Found - event not found.
PUT	/api/categories/{id}	Updates an existing category belonging to an event owned by the logged-in Organiser.	Organiser	{ categoryName, minimumAge, maximumAge, distance }	200 OK - category updated. 400 Bad Request - invalid data. 401 Unauthorized - not logged in. 403 Forbidden - not permitted. 404 Not Found - category not found.
DELETE	/api/categories/{id}	Deletes an existing category belonging to an event owned by the logged-in Organiser.	Organiser	None	204 No Content - category deleted. 401 Unauthorized - not logged in. 403 Forbidden - not permitted. 404 Not Found - category not found.

5. Event Enrolment Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/events/{eventId}/enrolments	Enrols the logged-in Participant into the selected event and category. ParticipantId comes from the logged-in account and EventId comes from the route.	Participant	{ categoryId }	201 Created - enrolment created. 400 Bad Request - invalid enrolment/category. 401 Unauthorized - not logged in. 403 Forbidden - user is not a Participant. 404 Not Found - event or category not found. 409 Conflict - participant already enrolled.
GET	/api/enrolments/my-enrolments	Returns all event enrolments for the currently logged-in Participant.	Participant	None	200 OK - own enrolments returned. 401 Unauthorized - user is not logged in. 403 Forbidden - user is not a Participant.
GET	/api/events/{eventId}/enrolments	Returns all enrolments for an event owned by the logged-in Organiser.	Organiser	None	200 OK - event enrolments returned. 401 Unauthorized - not logged in. 403 Forbidden - organiser does not own the event. 404 Not Found - event not found.

6. Result Endpoints

HTTP Method	Route	Description	Role Required	Request Body	Expected Response

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{enrolmentId}/results	Captures the finish time and finishing position for a Participant after an event.	Organiser	{ finishTime, finishPosition }	201 Created - result captured. 400 Bad Request - invalid result data. 401 Unauthorized - not logged in. 403 Forbidden - organiser is not permitted to manage this event. 404 Not Found - enrolment not found. 409 Conflict - result already exists for the enrolment.
GET	/api/results/my-results	Returns only the race results belonging to the currently logged-in Participant.	Participant	None	200 OK - own results returned. 401 Unauthorized - user is not logged in. 403 Forbidden - user is not a Participant.

Implementation Alignment Notes

- OrganiserId should be taken from the authenticated Organiser when an event is created; it does not need to be supplied in the request body.
- ParticipantId should be taken from the authenticated Participant when an enrolment is created. EventId comes from the route and CategoryId comes from the request body.
- CreatedAt, EnrolmentDate and RecordedAt should be generated by the server/database rather than supplied by the user.
- A new enrolment can automatically start with a status such as Registered so the Participant does not need to send the Status field.
- Event type should be validated to the values run, walk or cycle, matching the Part 2 functional requirements.
- Organiser operations on events, categories, enrolments and results should be restricted to events owned by that Organiser.
- Participants must only be able to view their own enrolments and their own results.

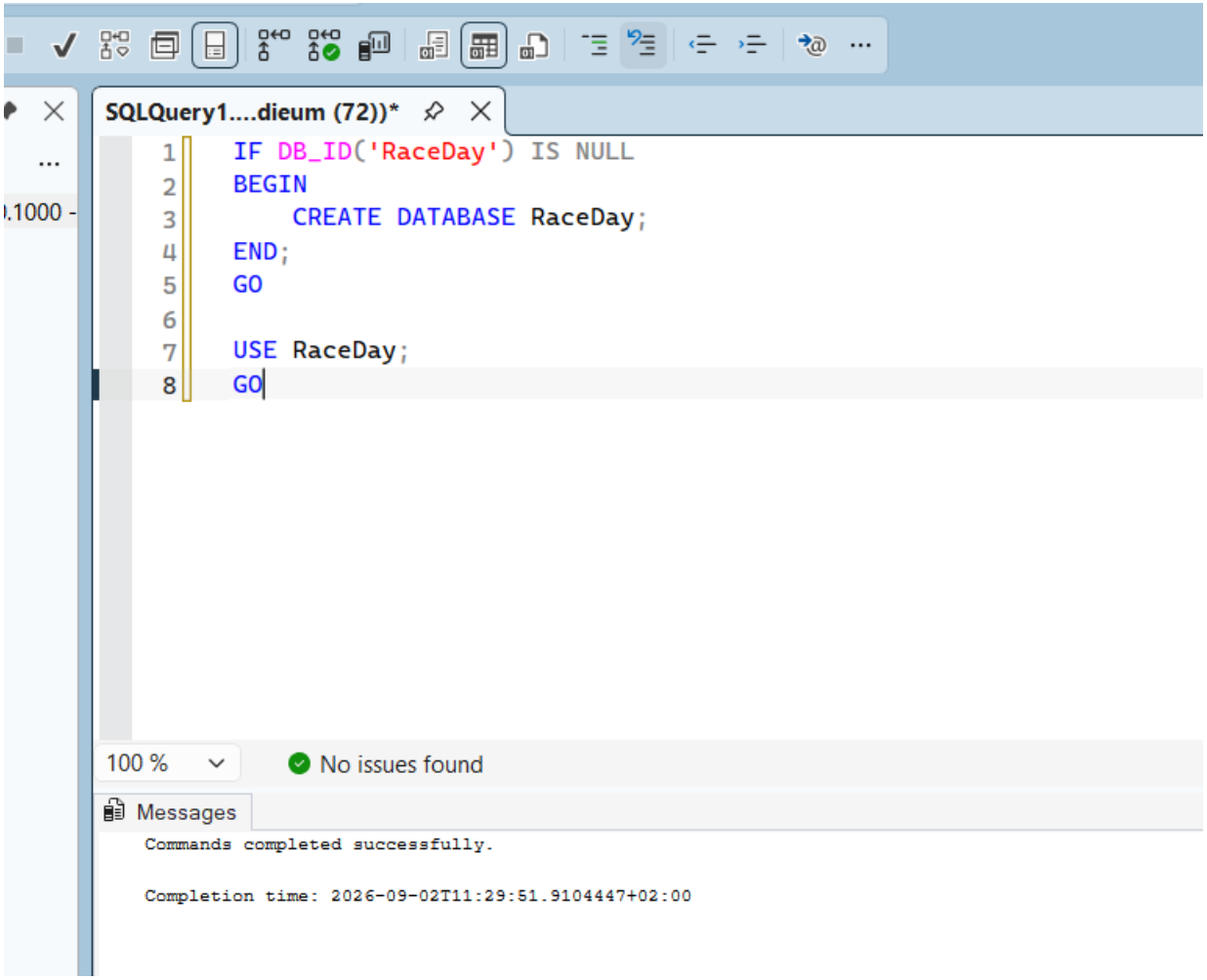
Conclusion: This revised Section B now matches the supplied Part 2 functional requirements and is intentionally limited to endpoints that are required or directly useful for implementing those requirements.

Section C – SQL DATABASE

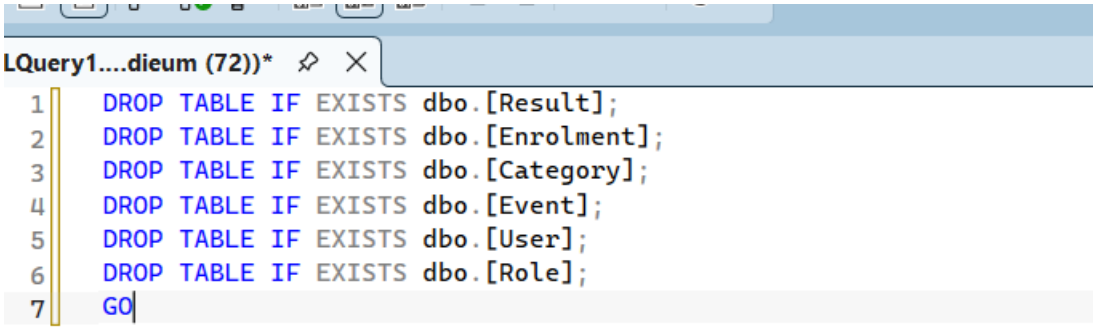
RaceDay System

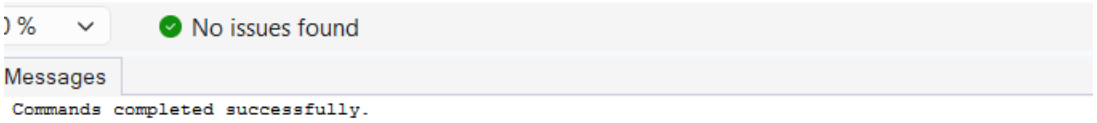
Screenshot of My SQL DATABASE:

1.

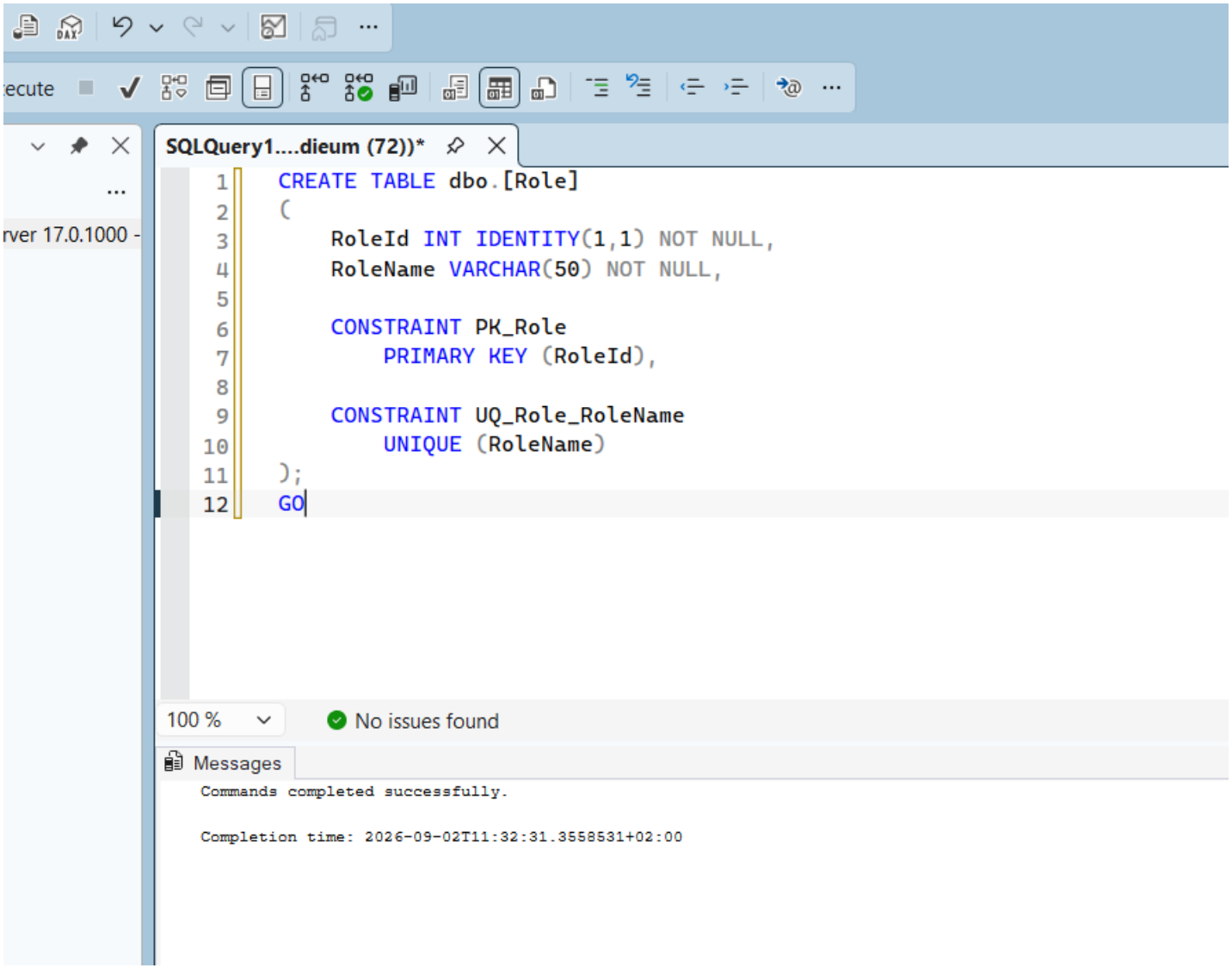


2.

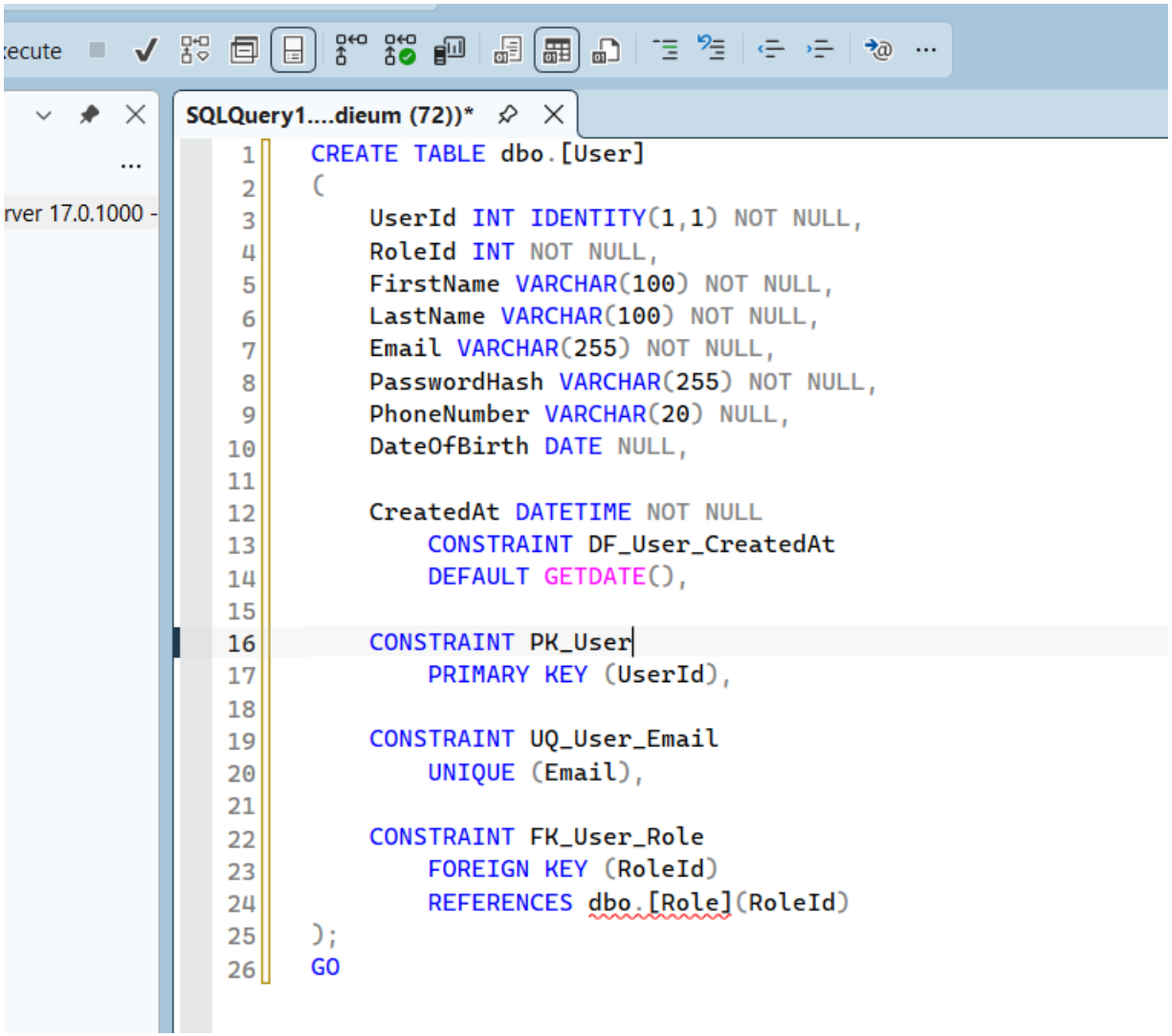

LQuery1....dieum (72))*
1 DROP TABLE IF EXISTS dbo.[Result];
2 DROP TABLE IF EXISTS dbo.[Enrolment];
3 DROP TABLE IF EXISTS dbo.[Category];
4 DROP TABLE IF EXISTS dbo.[Event];
5 DROP TABLE IF EXISTS dbo.[User];
6 DROP TABLE IF EXISTS dbo.[Role];
7 GO


) %
No issues found
Messages
Commands completed successfully.

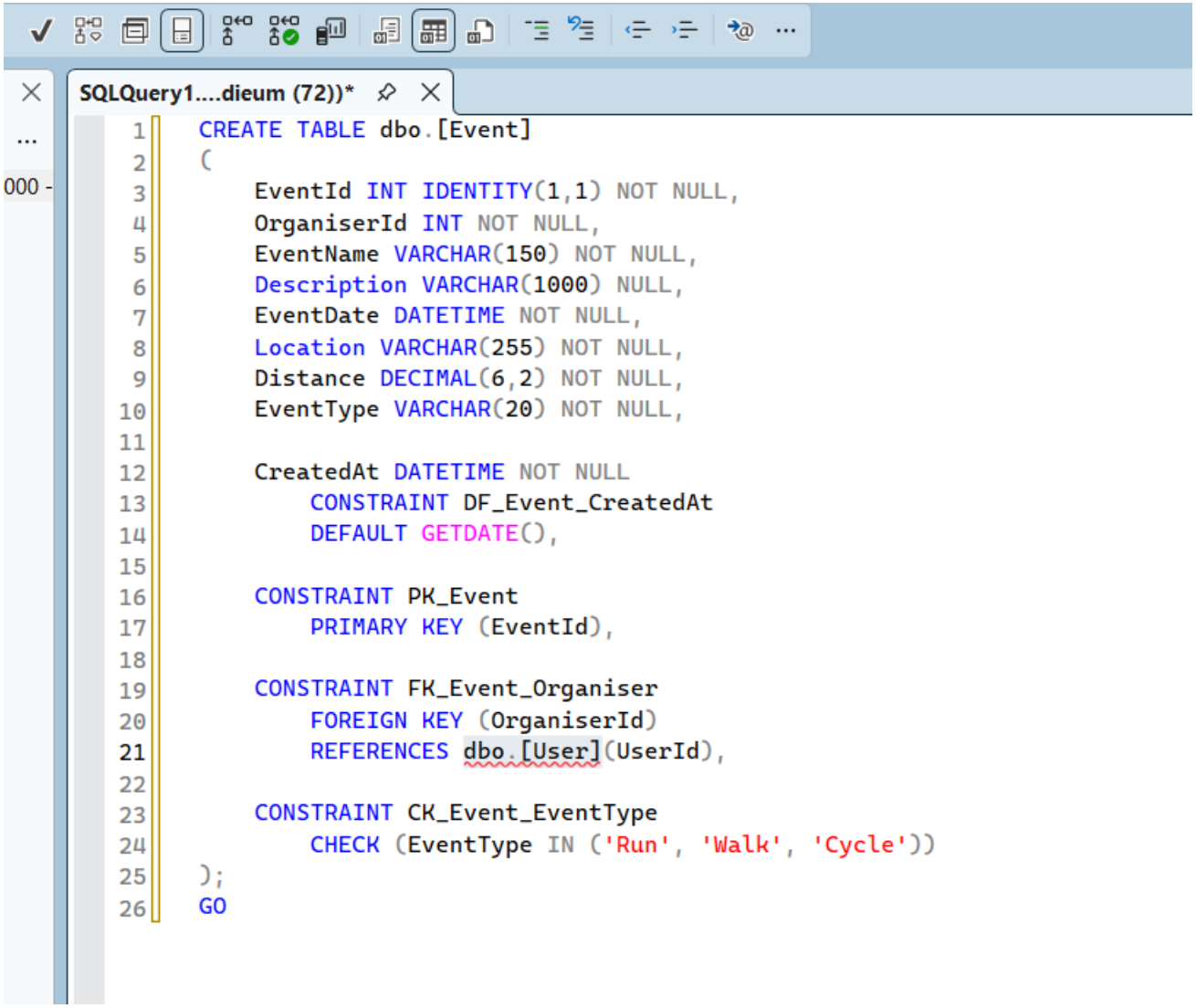
3.


SQLQuery1....dieum (72))*
CREATE TABLE dbo.[Role]
(
RoleId INT IDENTITY(1,1) NOT NULL,
RoleName VARCHAR(50) NOT NULL,
CONSTRAINT PK_Role
PRIMARY KEY (RoleId),
CONSTRAINT UQ_Role_RoleName
UNIQUE (RoleName)
);
GO
100 %
No issues found
Messages
Commands completed successfully.
Completion time: 2026-09-02T11:32:31.3558531+02:00

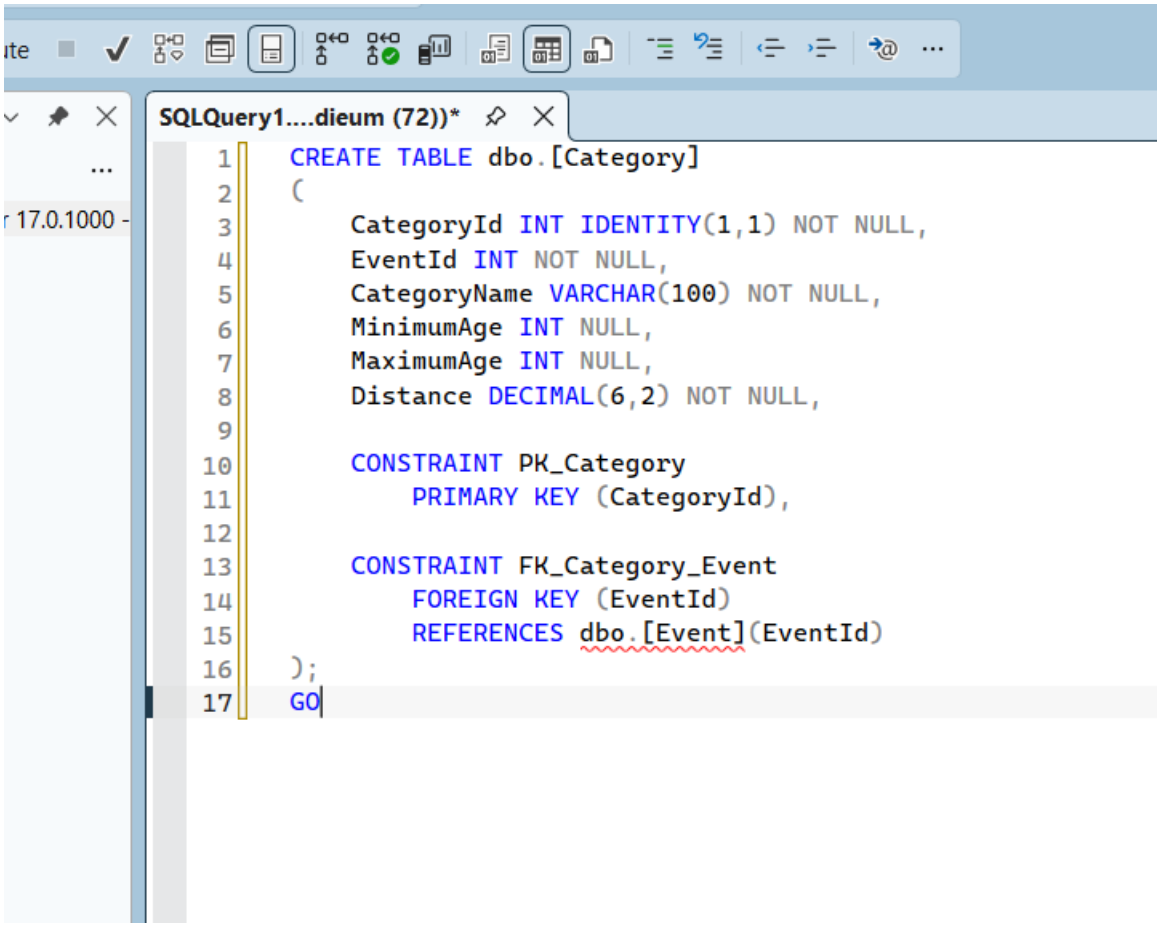
4.


SQLQuery1....dieum (72))*
CREATE TABLE dbo.[User]
(
UserId INT IDENTITY(1,1) NOT NULL,
RoleId INT NOT NULL,
FirstName VARCHAR(100) NOT NULL,
LastName VARCHAR(100) NOT NULL,
Email VARCHAR(255) NOT NULL,
PasswordHash VARCHAR(255) NOT NULL,
PhoneNumber VARCHAR(20) NULL,
DateOfBirth DATE NULL,
CreatedAt DATETIME NOT NULL
CONSTRAINT DF_User_CreatedAt
DEFAULT GETDATE(),
CONSTRAINT PK_User
PRIMARY KEY (UserId),
CONSTRAINT UQ_User_Email
UNIQUE (Email),
CONSTRAINT FK_User_Role
FOREIGN KEY (RoleId)
REFERENCES dbo.[Role](RoleId)
);
GO

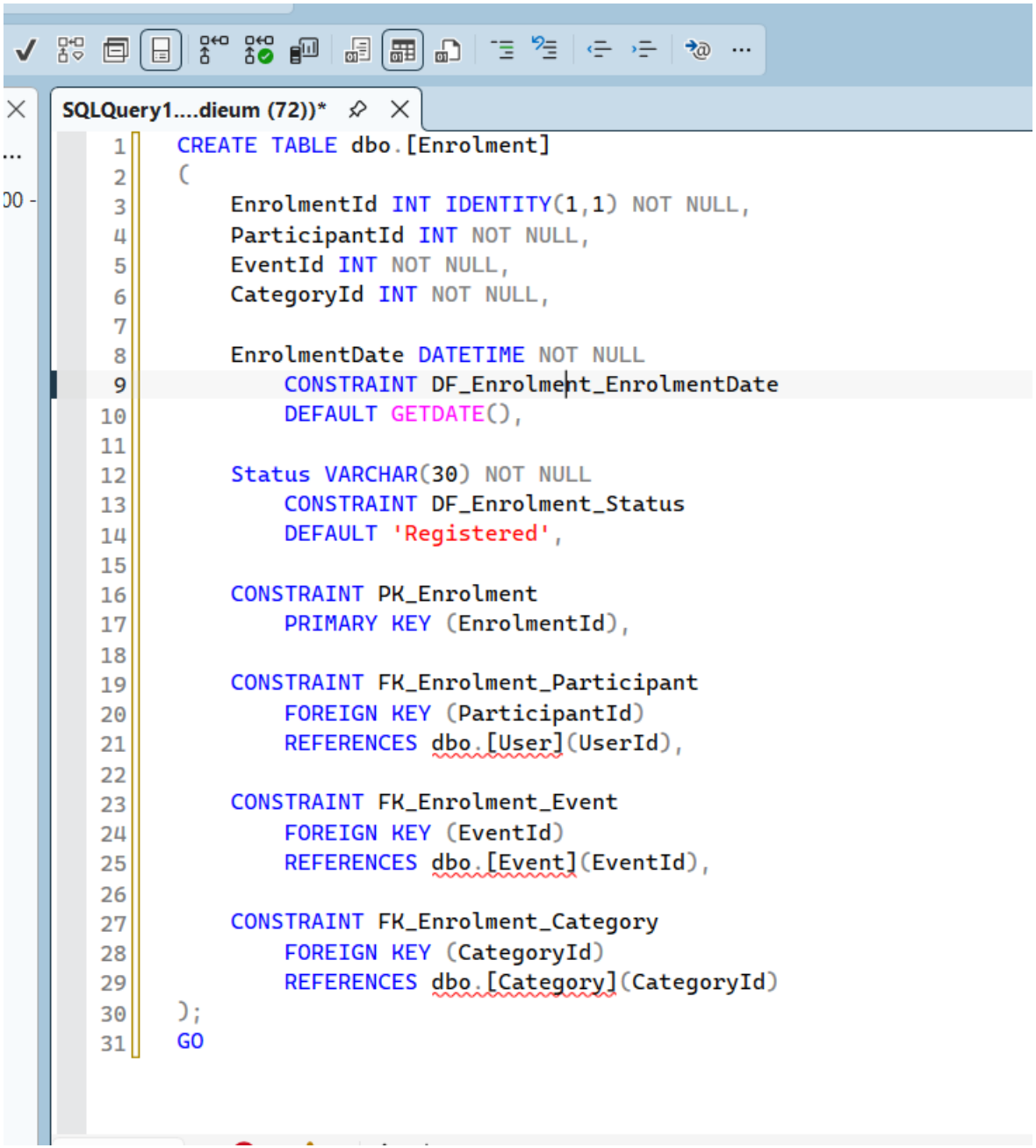
5.


SQLQuery1....dieum (72))*
CREATE TABLE dbo.[Event]
(
EventId INT IDENTITY(1,1) NOT NULL,
OrganiserId INT NOT NULL,
EventName VARCHAR(150) NOT NULL,
Description VARCHAR(1000) NULL,
EventDate DATETIME NOT NULL,
Location VARCHAR(255) NOT NULL,
Distance DECIMAL(6,2) NOT NULL,
EventType VARCHAR(20) NOT NULL,
CreatedAt DATETIME NOT NULL
CONSTRAINT DF_Event_CreatedAt
DEFAULT GETDATE(),
CONSTRAINT PK_Event
PRIMARY KEY (EventId),
CONSTRAINT FK_Event_Organiser
FOREIGN KEY (OrganiserId)
REFERENCES dbo.[User](UserId),
CONSTRAINT CK_Event_EventType
CHECK (EventType IN ('Run', 'Walk', 'Cycle'))
);
GO

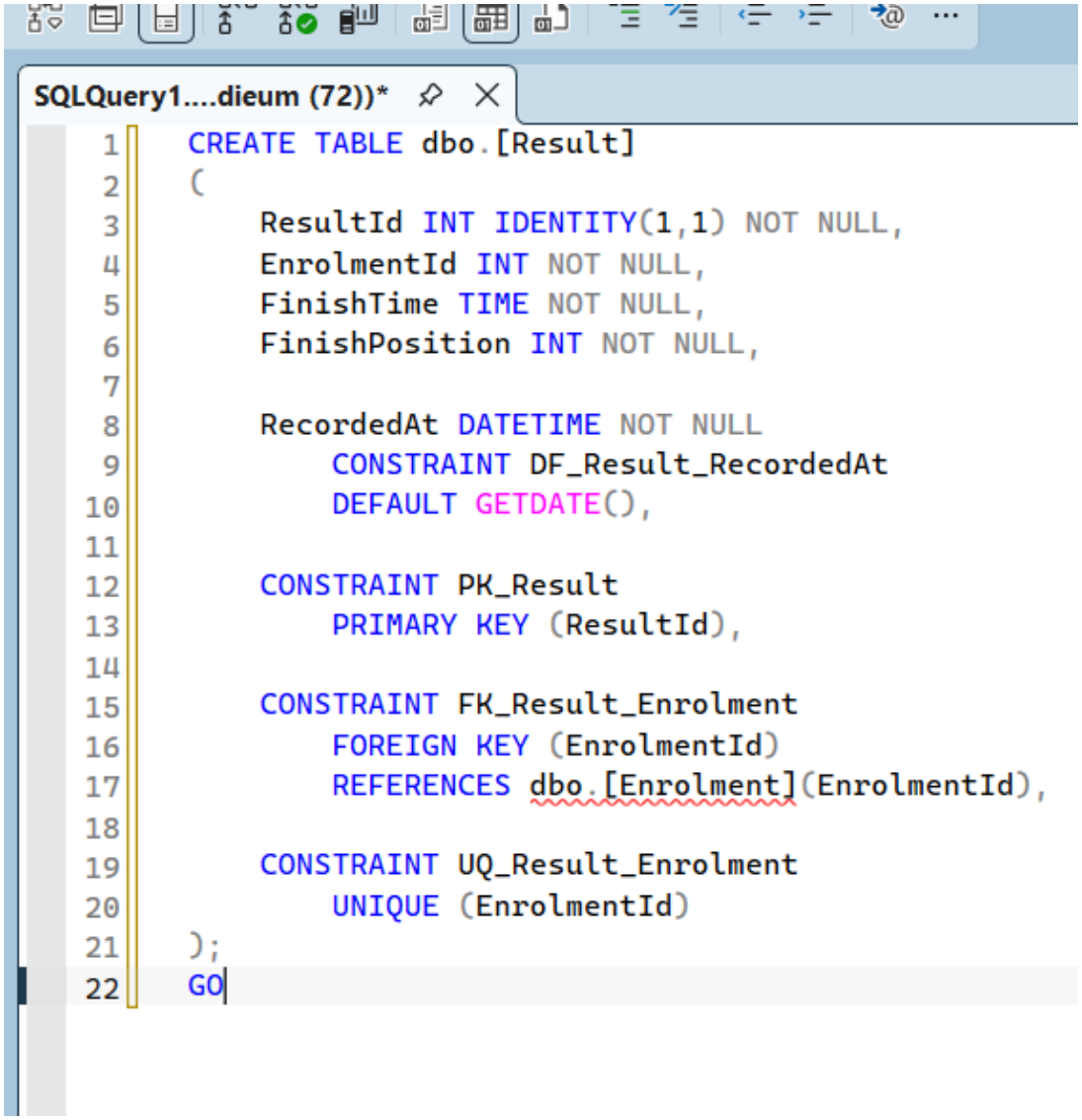
6.


SQLQuery1....dieum (72))*
17.0.1000 -
CREATE TABLE dbo.[Category]
(
CategoryId INT IDENTITY(1,1) NOT NULL,
EventId INT NOT NULL,
CategoryName VARCHAR(100) NOT NULL,
MinimumAge INT NULL,
MaximumAge INT NULL,
Distance DECIMAL(6,2) NOT NULL,
CONSTRAINT PK_Category
PRIMARY KEY (CategoryId),
CONSTRAINT FK_Category_Event
FOREIGN KEY (EventId)
REFERENCES dbo.[Event](EventId)
);
GO

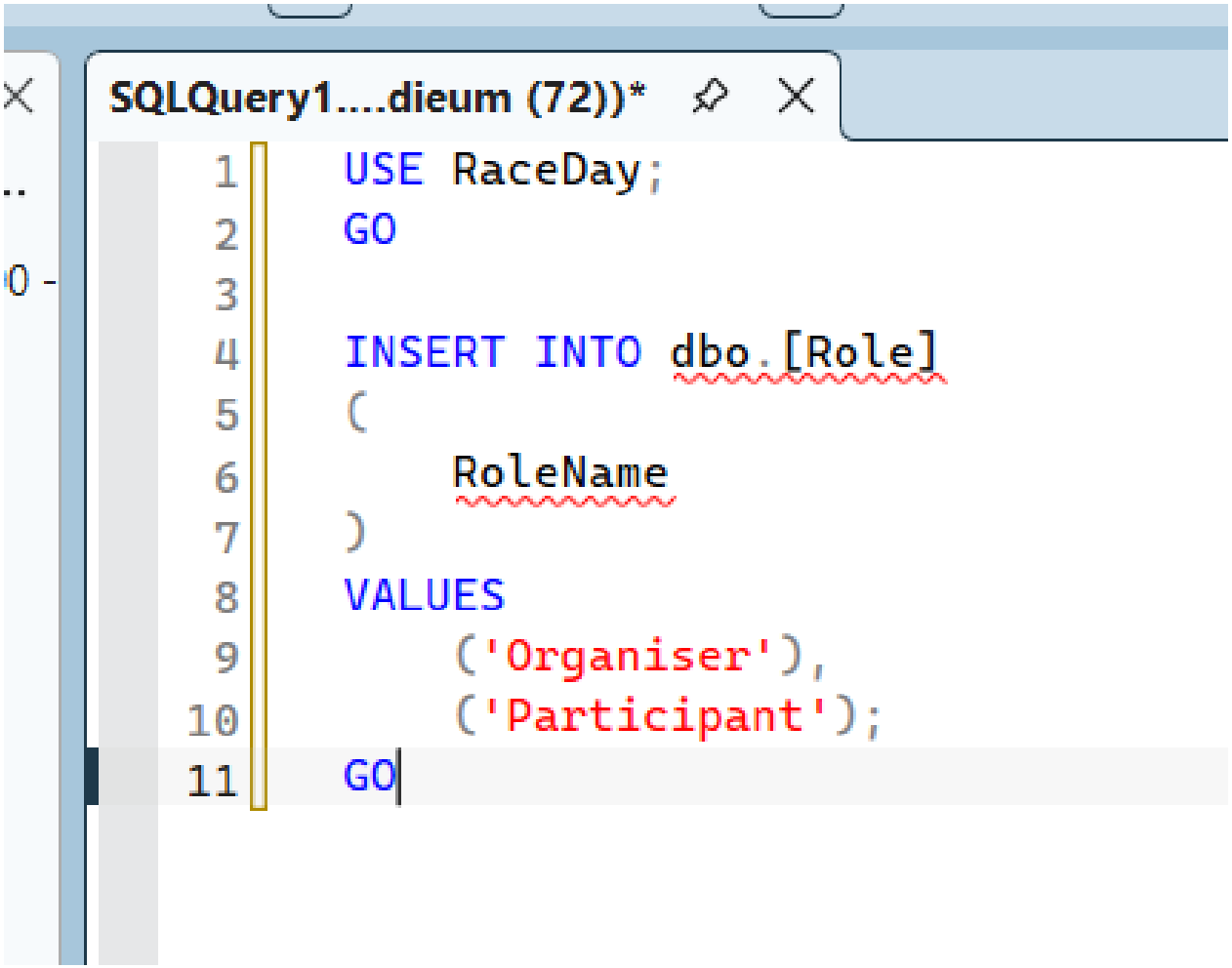
7.


SQLQuery1....dieum (72))*
CREATE TABLE dbo.[Enrolment]
(
EnrolmentId INT IDENTITY(1,1) NOT NULL,
ParticipantId INT NOT NULL,
EventId INT NOT NULL,
CategoryId INT NOT NULL,
EnrolmentDate DATETIME NOT NULL
CONSTRAINT DF_Enrolment_EnrolmentDate
DEFAULT GETDATE(),
Status VARCHAR(30) NOT NULL
CONSTRAINT DF_Enrolment_Status
DEFAULT 'Registered',
CONSTRAINT PK_Enrolment
PRIMARY KEY (EnrolmentId),
CONSTRAINT FK_Enrolment_Participant
FOREIGN KEY (ParticipantId)
REFERENCES dbo.[User](UserId),
CONSTRAINT FK_Enrolment_Event
FOREIGN KEY (EventId)
REFERENCES dbo.[Event](EventId),
CONSTRAINT FK_Enrolment_Category
FOREIGN KEY (CategoryId)
REFERENCES dbo.[Category](CategoryId)
);
GO

8.


SQLQuery1....dieum (72))*
CREATE TABLE dbo.[Result]
(
ResultId INT IDENTITY(1,1) NOT NULL,
EnrolmentId INT NOT NULL,
FinishTime TIME NOT NULL,
FinishPosition INT NOT NULL,
RecordedAt DATETIME NOT NULL
CONSTRAINT DF_Result_RecordedAt
DEFAULT GETDATE(),
CONSTRAINT PK_Result
PRIMARY KEY (ResultId),
CONSTRAINT FK_Result_Enrolment
FOREIGN KEY (EnrolmentId)
REFERENCES dbo.[Enrolment](EnrolmentId),
CONSTRAINT UQ_Result_Enrolment
UNIQUE (EnrolmentId)
);
GO

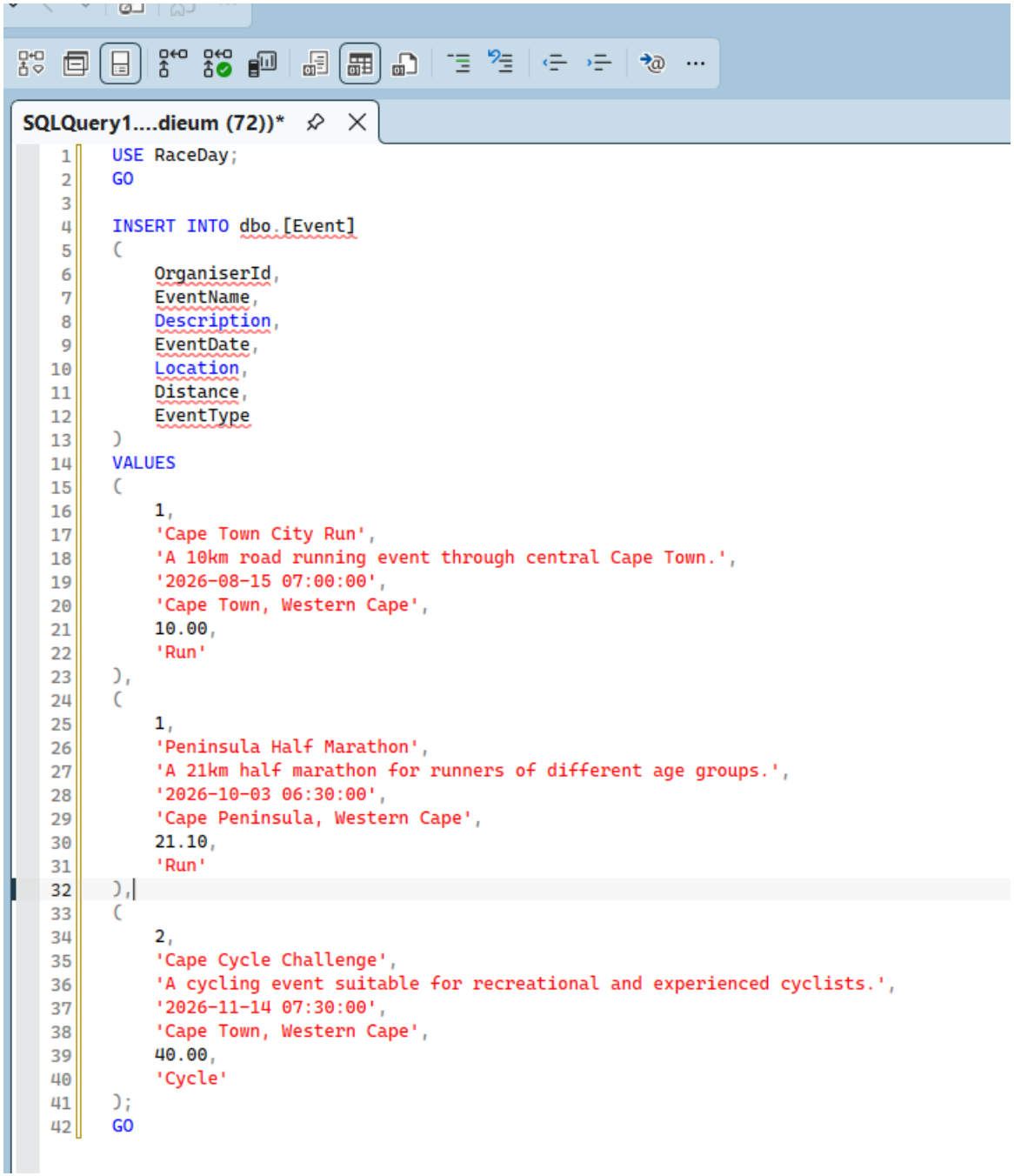
9.


SQLQuery1....dieum (72))*
USE RaceDay;
GO
INSERT INTO dbo.[Role]
(
RoleName
)
VALUES
('Organiser'),
('Participant');
GO

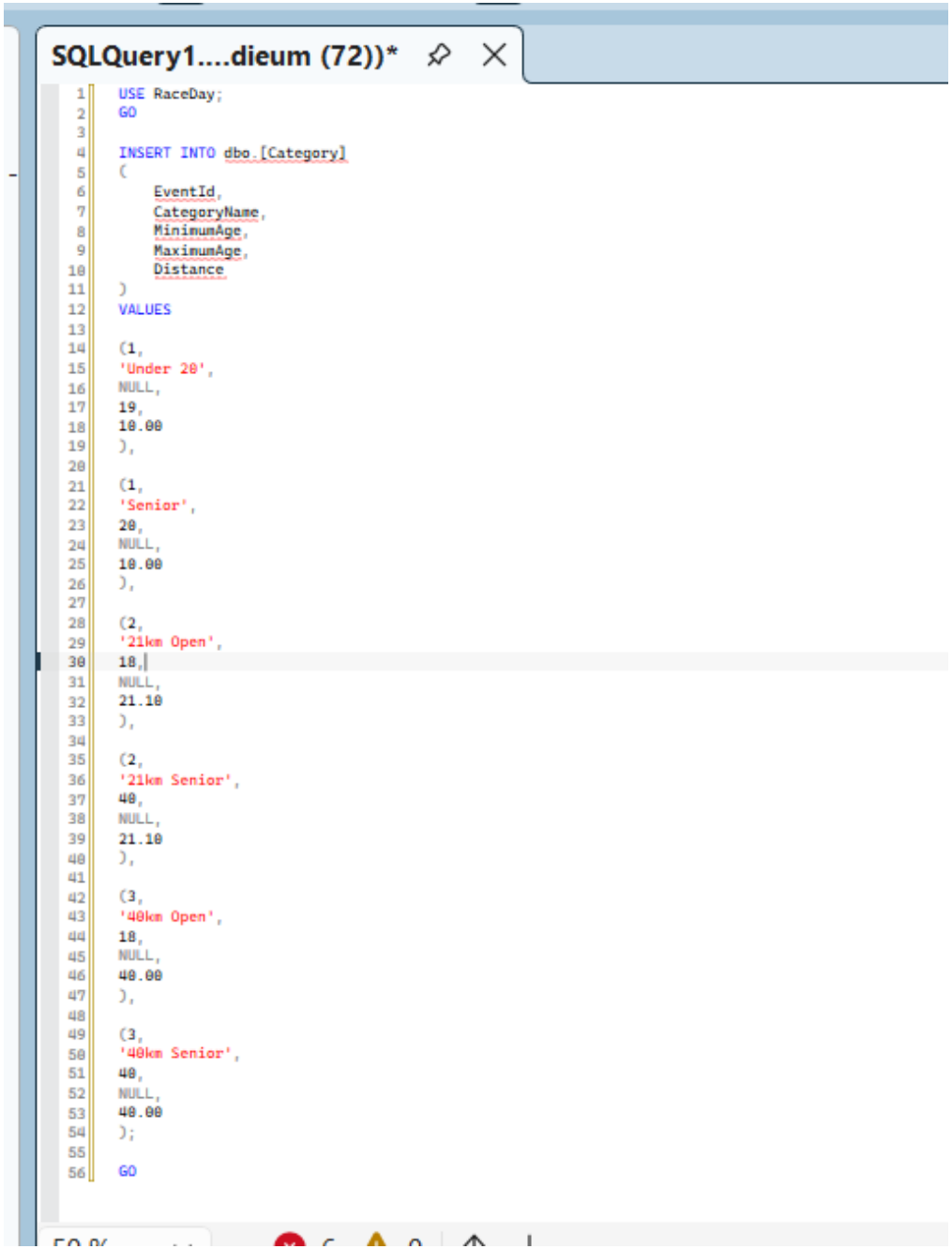
10.


SQLQuery1....dieum (72))*
USE RaceDay;
GO
INSERT INTO dbo.[User]
(
RoleId,
FirstName,
LastName,
Email,
PasswordHash,
PhoneNumber,
DateOfBirth
)
VALUES
(
1,
'Thandi',
'Ndlovu',
'thandi.ndlovu@raceday.co.za',
CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', 'RaceDay@123'), 2),
'0821234567',
'1988-05-15'
),
(
1,
'Michael',
'Jacobs',
'michael.jacobs@raceday.co.za',
CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', 'RaceDay@456'), 2),
'0832345678',
'1990-09-22'
),
(
2,
'Sipho',
'Dlamini',
'sipho.dlamini@email.com',
CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', 'Participant@123'), 2),
'0843456789',
'2001-03-10'
),
(
2,
'Ayesha',
'Khan',
'ayesha.khan@email.com',
CONVERT(VARCHAR(64), HASHBYTES('SHA2_256', 'Participant@456'), 2),
'0714567890',
'1997-11-25'
);
GO

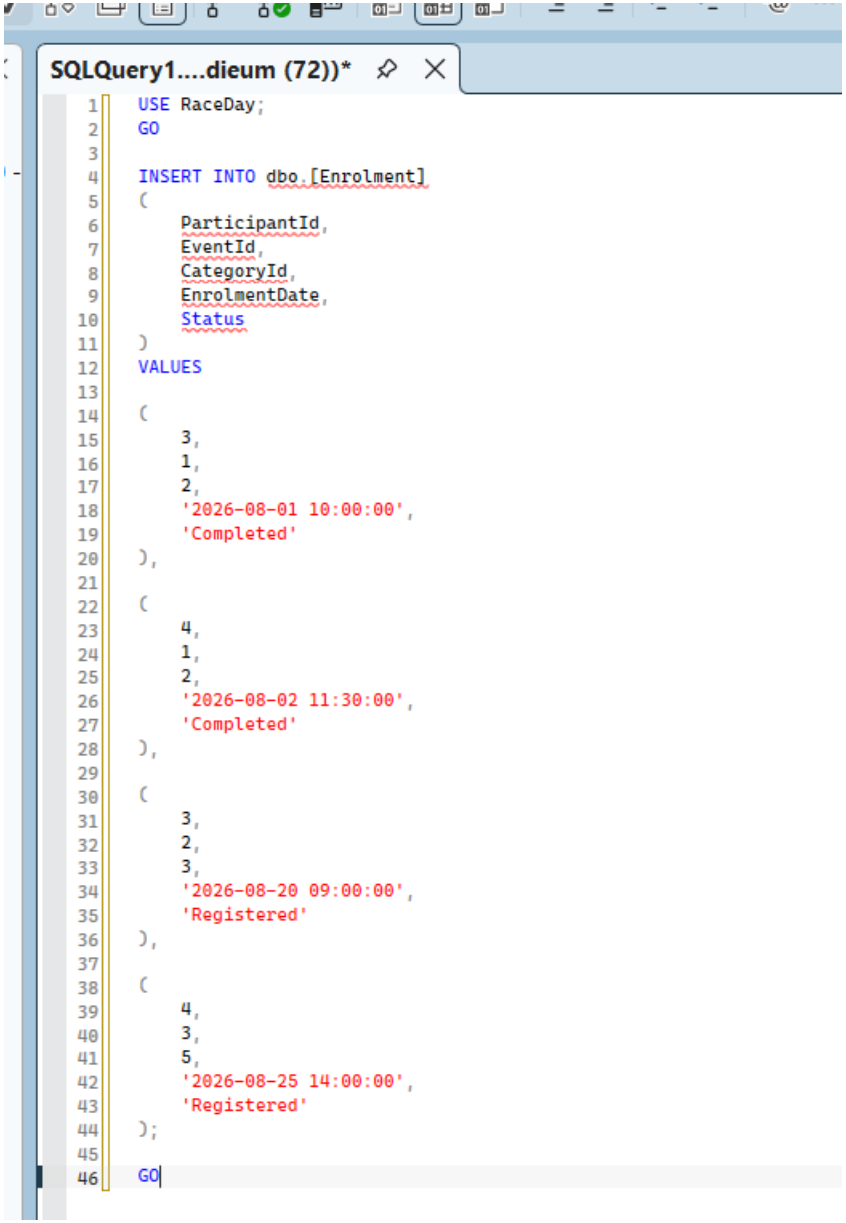
11.


SQLQuery1....dieum (72))*
USE RaceDay;
GO
INSERT INTO dbo.[Event]
(
OrganiserId,
EventName,
Description,
EventDate,
Location,
Distance,
EventType
)
VALUES
(
1,
'Cape Town City Run',
'A 10km road running event through central Cape Town.',
'2026-08-15 07:00:00',
'Cape Town, Western Cape',
10.00,
'Run'
),
(
1,
'Peninsula Half Marathon',
'A 21km half marathon for runners of different age groups.',
'2026-10-03 06:30:00',
'Cape Peninsula, Western Cape',
21.10,
'Run'
),
(
2,
'Cape Cycle Challenge',
'A cycling event suitable for recreational and experienced cyclists.',
'2026-11-14 07:30:00',
'Cape Town, Western Cape',
40.00,
'Cycle'
);
GO

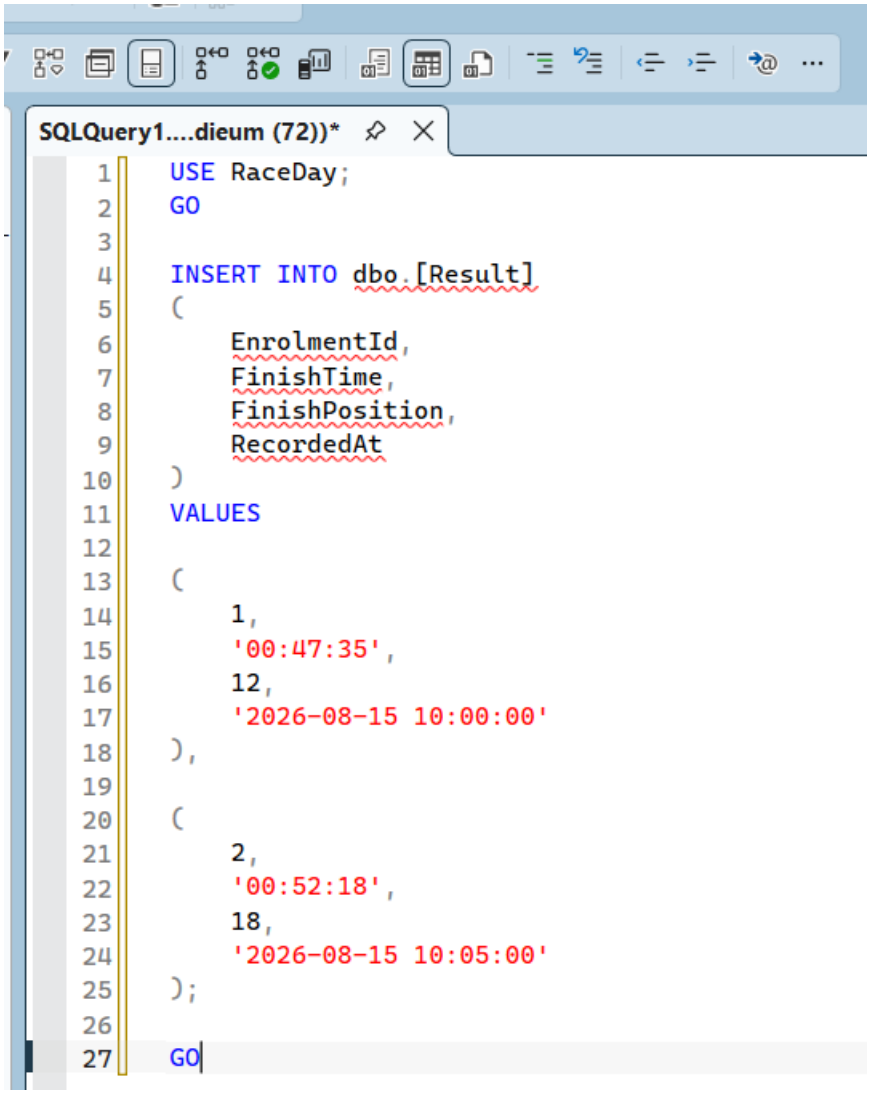
12.


SQLQuery1....dieum (72))*
USE RaceDay;
GO
INSERT INTO dbo.[Category]
(
EventId,
CategoryName,
MinimumAge,
MaximumAge,
Distance
)
VALUES
(1,
'Under 20',
NULL,
19,
10.00
),
(1,
'Senior',
20,
NULL,
10.00
),
(2,
'21km Open',
18,
NULL,
21.10
),
(2,
'21km Senior',
40,
NULL,
21.10
),
(3,
'40km Open',
18,
NULL,
40.00
),
(3,
'40km Senior',
40,
NULL,
40.00
);
GO

13.


SQLQuery1....dieum (72))*
USE RaceDay;
GO
INSERT INTO dbo.[Enrolment]
(
ParticipantId,
EventId,
CategoryId,
EnrolmentDate,
Status
)
VALUES
(
3,
1,
2,
'2026-08-01 10:00:00',
'Completed'
),
(
4,
1,
2,
'2026-08-02 11:30:00',
'Completed'
),
(
3,
2,
3,
'2026-08-20 09:00:00',
'Registered'
),
(
4,
3,
5,
'2026-08-25 14:00:00',
'Registered'
);
GO

14.



Note: I will also upload the SQL file on my github .